

Module 4: Control Structures

Introduction to C++ Programming — A Beginner's Course

Module Overview

This is the module where programs start to feel alive — they can decide and repeat. It's also the densest module so far (if/else, switch, three loop types, break/continue) and benefits from being split across two sessions if your schedule allows: decisions (if/switch) in one sitting, repetition (loops) in the next. The number guessing game project is usually the first program students describe as 'fun' rather than 'an exercise.'

Suggested Duration: 90 minutes total — recommend splitting into two 45-minute sessions (Decisions, then Loops) if possible

Learning Objectives

By the end of this module, students will be able to:

- Write correct comparison and logical expressions (==, !=, &&, ||, !).
- Build correctly ordered if / else if / else chains.
- Use switch appropriately, including remembering break in every case.
- Choose the appropriate loop (while, do-while, for) for a given repetition task.
- Use break and continue correctly inside a loop.
- Use the modulo operator (%) to test divisibility.

Prerequisites

- Modules 1–3: variables, types, and basic I/O.

Materials & Instructor Prep

- 04-control-structures/examples/ preloaded and ready to run in sequence.
- A visible countdown or stopwatch for live infinite-loop demonstrations (see below) so students know it's intentional, not a real crash.

Key Vocabulary

Term	Definition
Boolean expression	An expression that evaluates to true or false, used as a condition.
Branch	One possible path of execution chosen by if/else or switch.
Fall-through	In a switch, execution continuing into the next case because a break was omitted.
Iteration	One repetition of a loop's body.
Modulo (%)	The remainder-of-division operator; a % b is the remainder after dividing a by b.
Infinite loop	A loop whose condition never becomes false, so it never stops on its own.

Lesson Outline & Instructor Talking Points

SESSION A — Decisions

1. = vs. == — re-arm the warning from Module 3 (5 min)

- Write if (age = 18) on the board. Ask the class to predict what it does. Most beginners guess it checks equality.
- Reveal: it assigns 18 to age and the condition is always true (a non-zero value is 'truthy'), which is a real, silent, hard-to-spot bug class.

Instructor note: Many production compilers warn about this pattern — if your compiler doesn't show a warning by default, this is a good moment to enable -Wall and explain what warnings are for.

2. if / else if / else — order matters (20 min)

- Build the grade-lookup example live, but deliberately write it in the WRONG order first (smallest threshold first) and run it to show every high score incorrectly reports the lowest grade.
- Fix the ordering live and re-run. Make the rule explicit: conditions are checked top to bottom; the first true branch runs and the rest are skipped entirely.

Instructor note: This 'wrong order' demo is worth the extra 5 minutes — it's far more convincing than stating the rule and showing only the correct version.

3. switch, and the break footgun (15 min)

- Show the grade switch example. Remove one break live and run it, letting the class watch fall-through happen in real output.

- Frame switch as 'a cleaner if/else if chain for exact-value matches only' — not a replacement for range checks like `score >= 90`.

SESSION B — Repetition

4. while, do-while, for — same job, different shapes (25 min)

- Introduce while first as the most 'primitive' form: check condition, run body, repeat.
- Introduce do-while specifically through its motivating use case: input validation that must run the prompt at least once before it can check anything.
- Introduce for last, mapping its three clauses explicitly onto an equivalent while loop so students see for is not a new concept, just a compact notation for a very common pattern.

Instructor note: *Rewriting one for loop as an equivalent while loop live (and vice versa) is the fastest way to cement that these aren't three unrelated tools.*

5. The controlled infinite loop demo (10 min)

- Deliberately write a while loop that forgets to update its counter, run it, and let it hang — then press Ctrl+C on camera/projector and explain what just happened.
- This converts a scary real-world moment ('my program froze!') into a recognized, nameable, fixable situation before a student hits it alone at 11pm doing homework.

Instructor note: *Warn students in advance that you're about to do this on purpose, so it reads as a demo, not a mistake.*

6. Nested loops, break, continue, and % (20 min)

- Build the multiplication grid live with nested for loops; pause after the outer loop's first full inner-loop pass to ask 'what happens next?' before continuing.
- Introduce % explicitly as 'is it evenly divisible?' and tie it directly into the FizzBuzz-style exercise coming up.

Common Student Misconceptions

Misconception	How to Address It
"switch can check ranges like <code>score >= 90</code> ."	switch only matches exact values — range checks require if/else if. Show the compiler error if attempted.
"A for loop is fundamentally different from a while loop."	Rewrite one as the other live — a for loop is a while loop with its setup/condition/update gathered into one line.

Misconception	How to Address It
"break and continue do the same thing."	break exits the loop entirely; continue only skips the rest of the current iteration and keeps looping.

Exercises & Module Project

Exercise: 04-control-structures/exercises/exercise1.cpp — a FizzBuzz-style loop exercise. Project, Part A: a number guessing game with a loop and guess counter (04-control-structures/project/README.md). Project, Part B: revisiting the Module 3 BMI calculator to add real if/else if category detection.

Facilitation guidance: Part B of the project is pedagogically important, not optional filler — it's the first time students see a program they already built get meaningfully smarter using only new syntax, no new data. Call this connection out explicitly.

Discussion & Comprehension Check Questions

1. Why does the order of else if conditions matter, but the order of switch cases usually doesn't?

Guidance: if/else if stops at the first TRUE condition and ranges can overlap, so order changes behavior; switch cases match one exact value each, so (barring fall-through) order is usually irrelevant to which case fires.

2. Give an example of a loop where you know the exact number of iterations in advance, and one where you don't.

Guidance: Known count → for (e.g. printing 1 to 10); unknown count → while (e.g. reading input until the user types 'quit').

Assessment Ideas

- Whiteboard trace: give a small nested-loop snippet and have students write out its exact printed output by hand before running it.
- Bug hunt: provide a switch statement missing one break and ask students to predict the (wrong) output.

Answer Key Notes

See 04-control-structures/exercises/solutions/exercise1_solution.cpp.

Wrap-Up & Bridge to Next Module

Bridge forward: "Right now, every score or guess your program handles lives in its own separate variable. Next module, we learn to hold many related values together — arrays, vectors, and structs — which is what real data actually looks like."

Instructor Tips

- This module has the highest new-syntax density in the course. If you must cut something for time, cut do-while depth (it's the least-used of the three loop forms going forward) before cutting the ordering/fall-through demos.